NexusCRM: Implementation Project Plan

Student Name: Chuma Nxazonke

Student Number: 219181187

Submission Date: 21/09/2025

Instructor: Mr Temuso Makhurane

Module Code: PFD473S

Module: Professional Development

Advanced Diploma in ICT in Application Development.

Table of Contents

1.0 Introduction

1.1 Project Overview

This document is the Software Project Management Plan (SPMP) for the NexusCRM project. The project entails the design, development, testing, and deployment of a bespoke Customer Relationship Management (CRM) system for Visionary Media Inc., a mid-sized marketing agency. The new system will replace their current disparate set of spreadsheets and legacy software, aiming to centralize client data, automate campaign tracking, and streamline sales pipelines.

1.2 Scope, Objectives, and Participants (Project Charter Summary)

- **Scope:** Includes requirements gathering, UI/UX design, software development (front-end and back-end), data migration from old systems, user acceptance testing (UAT), training, and deployment. Out of Scope: Development of advanced AI features, integration with accounting software (Phase 2), and hardware provisioning.

- **Objectives:**
    1. Deliver a fully functional CRM system within 6 months and a budget of R1,250,000.
    2. Improve sales team efficiency by 25% through automated task management.
    3. Achieve a 99% data accuracy rate post-migration.
    4. Receive a 90%+ satisfaction rate from end-users post-training.
- **Key Participants:**
    - **Sponsor:** CEO, Visionary Media Inc.
    - **Project Manager:** Chuma Nxazonke
    - **Project Team:** 7 members.
    - **End-Users:** Sales, Marketing, and Customer Support teams at Visionary Media.

1.3 Evolution of this Plan

This SPMP will be updated and revised as the project progresses. Major changes will require formal approval from the project sponsor.

2.0 Project Organization

2.1 Strategic & Tactical Management (Team Organization & Rationale)

The project will use a Matrix Organizational Structure. Team members are drawn from different functional departments (e.g., Development, QA) but report directly to the Project Manager for the duration of this project. This structure is chosen because:

- **Rationale:** It allows for efficient use of specialized resources (e.g., one DB expert for multiple projects) while maintaining clear lines of authority and communication for the project manager, ensuring focus on project goals.

2.2 Team Members, Roles, and Duties (Human Resources Management)

Name	Role	Key Duties & Rationale
C. Nxazonke	Project Manager	Overall planning, budgeting, scheduling, reporting, and stakeholder management. **Rationale:** Requires strong leadership and PMP skills.
A. Smith	Business Analyst	Elicit requirements, create user stories, act as liaison between business and tech teams. **Rationale:** Strong communication skills.
D. Lee	UI/UX Designer	Design wireframes, prototypes, and final user interfaces. **Rationale:** Expertise in creating user-friendly designs.
B. Patel	Back-end Developer	Develop server, application, and database logic. **Rationale:** Expert in Python/Django and database architecture.
F. Khan	Front-end Developer	Develop the user-facing side of the web application. **Rationale:** Expert in React.js and JavaScript.
T. Ngwenya	QA/Test Engineer	Develop test plans, execute manual and automated tests. **Rationale:** Meticulous attention to detail.
S. van der Merwe	DevOps/Systems Engineer	Manage deployment, CI/CD pipeline, and server configuration. **Rationale:** Expertise in Azure/AWS and Docker.

3.0 Managerial Process Plans

3.1 Project Selection/Justification

The current manual processes lead to an estimated 15% loss in productivity due to data duplication and errors. The new CRM is expected to:

- **Increase Sales Conversion:** Streamlined pipeline expected to increase conversions by 10%.
- **Reduce Admin Time:** Automating reports saves 20 hours/week across the team.
- **Monetary Benefit:** The estimated annual ROI is R850,000 from increased sales and saved labour costs. The project cost of R1.25m will be repaid in under 18 months.

3.2 Lifecycle Model

The project will use an Agile-Waterfall Hybrid (Iterative) Model.

- **Why?** The high-level requirements are known (justifying a Waterfall-like initial plan), but detailed user needs will evolve (requiring Agile flexibility).
- **Explanation:** The project will be broken into 4 major phases (like Waterfall): Planning, Design, Development, Deployment. However, the Development phase will consist of four 3-week sprints (like Agile), each producing a working increment of the software for review.

3.3 Work Breakdown Structure (WBS), Estimates, and Cost Assessment WBS (Level 1 & 2):

1. Project Management
2. Requirements Analysis
3. System Design (UI/UX, Architecture)
4. Development
   4.1 Sprint 1: User Authentication & Core Profile Module
   4.2 Sprint 2: Sales Pipeline Module
   4.3 Sprint 3: Marketing Campaign Tracking Module
   4.4 Sprint 4: Reporting Dashboard & Integration
5. Testing (Unit, Integration, System, UAT)
6. Data Migration
7. Training & Deployment

Effort Estimation (Using Bottom-Up Estimating):
We'll estimate one sample task in man-hours and extrapolate.

- Task: Develop "Add New Client" Feature (Part of 4.1)
    - Back-end (API, DB): 8 hours
    - Front-end (Form, Validation): 12 hours
    - Code Review & Integration: 2 hours
    - Total: 22 man-hours

Multiplying this by the number of features gives a total estimated effort. For this project, the total effort is estimated at 2,500 man-hours.

Resource Allocation & Cost Assessment:

Role	Hourly Rate (ZAR)	Estimated Hours	Total Cost (ZAR)
Project Manager	R650	300	R195,000
Business Analyst	R500	200	R100,000
UI/UX Designer	R450	150	R67,500
Back-end Developer	R550	700	R385,000
Front-end Developer	R550	600	R330,000
QA Engineer	R500	350	R175,000
DevOps Engineer	R600	200	R120,000
Subtotal (Labour)			R1,372,500
Contingency (10%)			R137,250
Software/Licenses			R90,000
Training/Misc.			R50,250
TOTAL PROJECT BUDGET			R1,650,000

3.4 Risk Management

Risk Management is a proactive process essential for improving project performance by systematically identifying, appraising, and managing project-related uncertainty. It extends beyond conventional planning to influence the project's design and execution actively. Our approach considers both threats (downside risks) and opportunities (upside potential) to ensure the NexusCRM project is resilient and can capitalize on favourable possibilities.

3.4.1 Risk Management Process

The project will follow a continuous, iterative risk management cycle:

1. **Risk Identification:** Continuously engaging all team members and stakeholders to brainstorm and document potential sources of uncertainty. This includes technology, people, management processes, and external factors like the political environment or contract issues.
2. **Risk Analysis:** Assessing each identified risk based on its probability of occurrence and potential impact on project objectives (cost, time, scope, quality).

3. **Risk Prioritization:** Ranking risks to focus management attention on the most significant items, typically those with high probability and high impact.
4. **Risk Planning (Response Planning):** Developing specific strategies to address each priority risk.
5. **Risk Monitoring & Control:** Continuously tracking identified risks, watching for trigger symptoms, and executing the response plans as needed.

3.4.2 Risk Assessment and Response Strategies

Key risks have been identified and assessed. The following table details the top risks, their mitigation (for threats) or exploitation (for opportunities) strategies, and assigns ownership.

Risk	Likelihood	Impact	Mitigation/Exploitation Strategy	Owner
Scope Creep	High	High	**Avoidance:** Implement a strict change control process. All changes require formal impact analysis and sponsor approval.	Project Manager
Key Team Member Leaves	Medium	High	**Mitigation:** Ensure knowledge sharing via pair programming and detailed documentation. Cross-train team members on critical components.	Project Manager
Unrealistic Schedule Estimates	High	Medium	**Mitigation:** Include a 15%-time buffer. Use Agile sprints to re-prioritize work regularly. **Exploitation:** If a task finishes early, use the time for risk reduction activities (e.g., refactoring, additional testing).	Project Manager
Data Migration Failure	Medium	Critical	**Mitigation:** Develop a robust, tested migration script. Perform multiple test migrations on a copy of the live data before the final cut-over.	DevOps Engineer
Technical Debt/Quality Shortfalls	High	Medium	**Mitigation:** Enforce code reviews, mandate 80%-unit test coverage, and use continuous integration to catch defects early.	Tech Lead

Opportunity: New Tech Improves Efficiency	Low	High	**Exploitation:** Dedicate a small research spike in a sprint to evaluate a promising new tool or library. If it proves beneficial, plan its adoption.	Tech Lead

3.4.3 Risk Monitoring and Control

The Risk Register will be a live document, updated and reviewed in weekly team meetings.

- **Symptoms Monitoring:** For each top risk, specific symptoms will be monitored (e.g., for "Unrealistic Schedule," symptom = tasks consistently exceeding estimates by >20%).
- **Metrics:** Key metrics such as schedule variance, defect density, and the rate of new change requests will be tracked to indicate overall project stability and risk exposure.
- **Contingency Plans:** The contingency budget (10%) and time buffer are reserved specifically for executing risk response plans should triggers be met.

This structured approach ensures that uncertainty is managed proactively, protecting the project's objectives while remaining agile enough to exploit opportunities for added benefit.

3.5 Quality Management

Quality Management for the NexusCRM project is defined as the totality of features and characteristics that bear on its ability to satisfy stated and implied needs. This encompasses not only delivering the required functionality (functional requirements) but also ensuring critical quality attributes such as efficiency, usability, dependability, and maintainability. Our approach is based on the ISO framework, separating quality activities into distinct but interconnected processes: planning, assurance, and control.

3.5.1 Quality Planning

During project initiation, applicable procedures and standards were selected and tailored for this project. The quality plan is a succinct document that defines:

- **Quality Goals:** Measurable targets for the product, including a target defect density of < 0.05 defects per function point and a target mean time between failures (MTBF) of > 120 hours during UAT.
- **Relevant Standards:** Adherence to international coding standards and organizational documentation standards.

- **Process Descriptions:** How quality activities (reviews, testing, audits) will be conducted.
- **Risks and Risk Management:** Identification of risks to product quality and mitigation strategies.

This plan ensures a shared understanding of quality expectations and provides a framework for all subsequent quality activities.

3.5.2 Quality Assurance (QA)

Quality Assurance establishes the organizational procedures and standards for quality and evaluates overall project performance to provide confidence that these standards are being followed. Key activities include:

- **Process Standards:** Defining how reviews should be conducted, how configuration management is enacted, and the required documentation process.
- **Quality Audits:** Independent, systematic examinations performed by an internal or external auditor to assess the adequacy of system controls and ensure compliance with established quality policies and procedures. Audits will be conducted during the project, not just at its end, to provide timely insights and recommendations for improvement.
- **Training:** Ensuring all team members receive the necessary training to understand and implement the defined quality processes effectively.

3.5.3 Quality Control (QC)

Quality Control involves the operational techniques and activities used to monitor specific project results to ensure they comply with relevant quality standards. For the NexusCRM project, this involves two primary approaches:

1. **Quality Reviews:** Formal, documented, and systematic examinations of work products. This includes:
    - **Design Reviews:** To evaluate the design's capability to meet requirements before implementation.
    - **Code Inspections:** For defect removal, focusing on identifying anomalies and ensuring adherence to coding standards.
    - **Test Plan Reviews:** To validate the completeness of the testing strategy.
    - Review results will be classified as "No Action," "Refer for Repair," or "Reconsider Overall Design," with all decisions and rationales recorded.

2. **Automated Software Assessment & Measurement:** Continuous evaluation using defined software metrics to quantify the software and the software process. Key metrics include:

    - **Product Metrics:** Defect density, mean time to failure (MTTF), and test coverage percentages to predict product attributes and identify problematic components.
    - **Process Metrics:** Tracking deviations from the schedule, resources expended, and the rate of change requests to measure project stability and technical progress.
    - **Customer Satisfaction Metrics:** To be gathered post-release via surveys monitoring parameters like usability, performance, and reliability.

3.5.4 Organizational Maturity

The project's quality processes are aligned with Level 2 (Repeatable) of the Capability Maturity Model (CMM), focusing on establishing fundamental project management controls. This is demonstrated through defined Key Process Areas (KPAs) such as:

- **Requirements Management:** Controlling requirements to establish a baseline.
- **Software Project Tracking:** Tracking actual results against plans and managing corrective actions.
- **Software Quality Assurance:** Planning SQA activities and objectively verifying adherence to procedures.
- **Software Configuration Management:** As defined in section 3.6.

This structured approach to quality management ensures that the NexusCRM system is not only built correctly but also delivers the required value and reliability to its users, thereby satisfying stated and implied needs.

3.6 Configuration Management

This section outlines the formal process for controlling and monitoring changes to all work products within the NexusCRM project. Once a baseline is established, configuration management minimizes risks by ensuring all changes are made consistently with project goals through a formal approval and tracking process.

3.6.1 Configuration Identification

All principal deliverables and components are identified as Configuration Items (CIs) once agreed upon. The NexusCRM project will treat the following as CIs:

- **Software Components:** Source code for each module (e.g., Authentication Module, Sales Pipeline Module).
- **Documentation:** The Software Project Management Plan (SPMP), Requirement Specifications, UI/UX Design Documents, and Technical Architecture Diagrams.
- **Build Tools:** Continuous Integration/Deployment (CI/CD) scripts and configuration files.

Each CI will be uniquely labelled using a three-digit version identification scheme: [CI Name]. [Major]. [Minor]. [Revision] (e.g., SalesPipeline.dll.1.2.0). A consistent set of versions of these CIs constitutes a configuration.

3.6.2 Change Control

A formal change control process will be implemented to ensure consistency with project goals. The process is initiated by a Change Request (CR), a formal report issued by any stakeholder to request a modification.

1. **Submission:** The CR author specifies the CI, its current version, the problem, and a proposed solution.
2. **Assessment:** The Project Manager, in consultation with technical leads, will assess the costs, benefits, and impact of the change.
3. **Approval:** For changes affecting scope, budget, or timeline, a Control Board comprising the Project Manager, Sponsor, and Tech Lead will approve or reject the request. The rationale for all decisions will be recorded.
4. **Implementation:** Upon approval, the change is assigned to a developer for implementation.

3.6.3 Versioning, Promotion, and Release Management

- **Version:** Identifies the state of a CI at a point in time.
- **Promotion:** A version made available to other developers. For example, a subsystem will be promoted to a Master Directory (controlled library) after passing unit tests, signalling it is stable for integration by other teams.
- **Release:** A version that has met quality criteria and is made available to users. A version is released to the Repository (static library) only after passing User Acceptance Testing (UAT). Beta releases may be made available to a limited user group for testing.

3.6.4 Workspace, Repository, and Library Management

The project will utilize a three-tiered structure to manage CIs:

1. **Workspace (Dynamic Library):** The developer's private environment for everyday development. Change is controlled by the individual developer.
2. **Master Directory (Controlled Library):** Stores promotions. To check in a version, it must meet project criteria (e.g., compiles without errors, passes unit tests).
3. **Repository (Static Library):** Stores releases. Promotions must meet strict quality control criteria (e.g., all regression tests pass) before being released.

3.6.5 Status Accounting and Auditing

- **Status Accounting:** The status of all CIs and change requests will be recorded and maintained. This allows developers to track the history of changes and enables management to have a real-time view of project status.
- **Auditing:** Prior to a release, the Quality Assurance (QA) team will perform an audit to validate the completeness, consistency, and quality of the product configuration. This ensures the release is built from the correct, approved versions of all CIs.

3.6.6 Tools

The project will use Git for version control, providing robust support for branching, merging, and labelling versions. GitHub will serve as the platform for the Master Directory and Repository, facilitating change tracking and collaboration. The CI/CD pipeline, integrated with GitHub, will automate the build and promotion process.

3.7 Rationale Management

All major decisions (e.g., technology stack selection, design choices) will be documented in a "Decision Log" (stored in Confluence). The log will include:

- Date of Decision
- Decision Made
- Alternatives Considered
- Rationale for Final Choice
- Person Responsible for the Decision

3.8 Procurement Management

Procurement is minimal for this software project. Required items:

- **Azure Cloud Server Subscription:** Will be procured through the company's IT department using an existing enterprise agreement.
- **Jira/Confluence Licenses:** Already available company-wide.
- **UI Design Tool (Figma):** Subscription to be purchased by the Project manager.

4.0 Technical Process Plans

This section of the SPMP would detail the development methods, technologies, etc. For brevity, I'll summarize.

- **Development Methodology:** This project uses a hybrid methodology. The high-level plan follows a sequential structure for clear milestones and budgeting. However, within the development phase, we employ an Agile-Scrum framework. The team will work in iterative, three-week sprints to build the software incrementally. This allows for flexibility to adapt to feedback while maintaining overall control of the project's timeline and objectives.
- **Technology Stack:** Front-end: React.js, Back-end: Python/Django, Database: PostgreSQL, Deployment: Docker on Azure App Service.

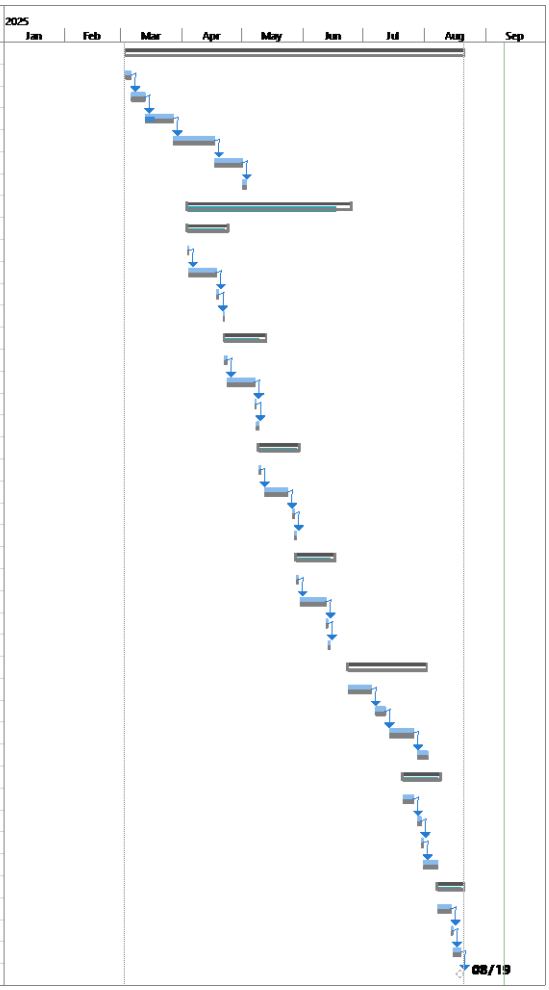
5.0 Appendices

Appendix A: Gantt Chart & Schedule

Overall Project Schedule: 6 Months (26 Weeks)

- Phase 1: Planning & Design (Weeks 1-5)
- Phase 2: Development Sprints (Weeks 6-17)
    - Sprint 1: Weeks 6-8
    - Sprint 2: Weeks 9-11
    - Sprint 3: Weeks 12-14
    - Sprint 4: Weeks 15-17
- Phase 3: Testing & Data Migration (Weeks 18-21)
- Phase 4: Training & Deployment (Weeks 22-24)
- Phase 5: Project Closure (Weeks 25-26)

ID	Task Mode	Task Name	Duration	Start	Finish
1		**Phase 1: Planning & Design**	**124 days?**	**2025/03/03**	**2025/08/20**
2		Project Kickoff & Charter Sign-Off	3 days?	2025/03/03	2025/03/05
3		Requirements Gathering Workshop	5 days?	2025/03/06	2025/03/12
4		Develop Functional Specifications	10 days?	2025/03/13	2025/03/26
5		UI/UX Design & Prototyping	15 days	2025/03/27	2025/04/16
6		Technical Architecture Design	10 days?	2025/04/17	2025/04/30
7		Phase 1 Review & Approval	2 days?	2025/05/01	2025/05/02
8		**Phase 2: Development Sprints**	**60 days?**	**2025/04/03**	**2025/06/24**
9		**Sprint 1: Auth & Core Profile**	**15 days?**	**2025/04/03**	**2025/04/23**
10		Sprint 1 Planning	1 day?	2025/04/03	2025/04/03
11		Active Development	10 days?	2025/04/04	2025/04/17
12		Sprint 1 Review & Demo	1 day?	2025/04/18	2025/04/18
13		Spring 1 Retrospective	1 day?	2025/04/21	2025/04/21
14		**Sprint2: Sales Pipeline**	**15 days?**	**2025/04/22**	**2025/05/12**
15		Sprint 2 Planning	1 day?	2025/04/22	2025/04/22
16		Active Development	10 days?	2025/04/23	2025/05/06
17		Sprint 2 Review & Demo	1 day?	2025/05/07	2025/05/07
18		Sprint 2 Retrospecive	1 day?	2025/05/08	2025/05/08
19		**Sprint 3: Campaign Tracking**	**15 days?**	**2025/05/09**	**2025/05/29**
20		Sprint 3 Planning	1 day?	2025/05/09	2025/05/09
21		Active Development	10 days?	2025/05/12	2025/05/23
22		Sprint 3 Review & Demo	1 day?	2025/05/26	2025/05/26
23		Sprint 3 Retrospective	1 day?	2025/05/27	2025/05/27
24		**Sprint 4: Reporting & Integration**	**15 days?**	**2025/05/28**	**2025/06/16**
25		Sprint 4 Planning	1 day?	2025/05/28	2025/05/28
26		Active Development	10 days?	2025/05/30	2025/06/11
27		Sprint Review & Demo	1 day?	2025/06/12	2025/06/12
28		Sprint 4 Retrospective	1 day?	2025/06/13	2025/06/13
29		**Phase 3: Testing & Data Migration**	**30 days?**	**2025/06/23**	**2025/08/01**
30		Performance & Security Testing	10 days?	2025/06/23	2025/07/04
31		Develop Data Migration Scripts	5 days?	2025/07/07	2025/07/11
32		User Acceptance Testing (UAT)	10 days?	2025/07/14	2025/07/25
33		Fix UAT Defects	5 days?	2025/07/28	2025/08/01
34		**Phase 4: Training & Deployment**	**15 days?**	**2025/07/21**	**2025/08/08**
35		End-User Training Session	5 days?	2025/07/21	2025/07/25
36		Final Data Migration	2 days?	2025/07/28	2025/07/29
37		System Go-Live Deployment	1 day?	2025/07/30	2025/07/30
38		Post-Launch Hypercare Support	5 days?	2025/07/31	2025/08/06
39		**Phase 5: Project Closeure**	**10 days?**	**2025/08/07**	**2025/08/20**
40		Final Project Report & Documentatio	5 days?	2025/08/07	2025/08/13
41		Project Retrospective with Team	1 day?	2025/08/14	2025/08/14
42		Client Sign-Off & Archiving	2 days?	2025/08/15	2025/08/18
43		Project End	0 days	2025/08/19	2025/08/19



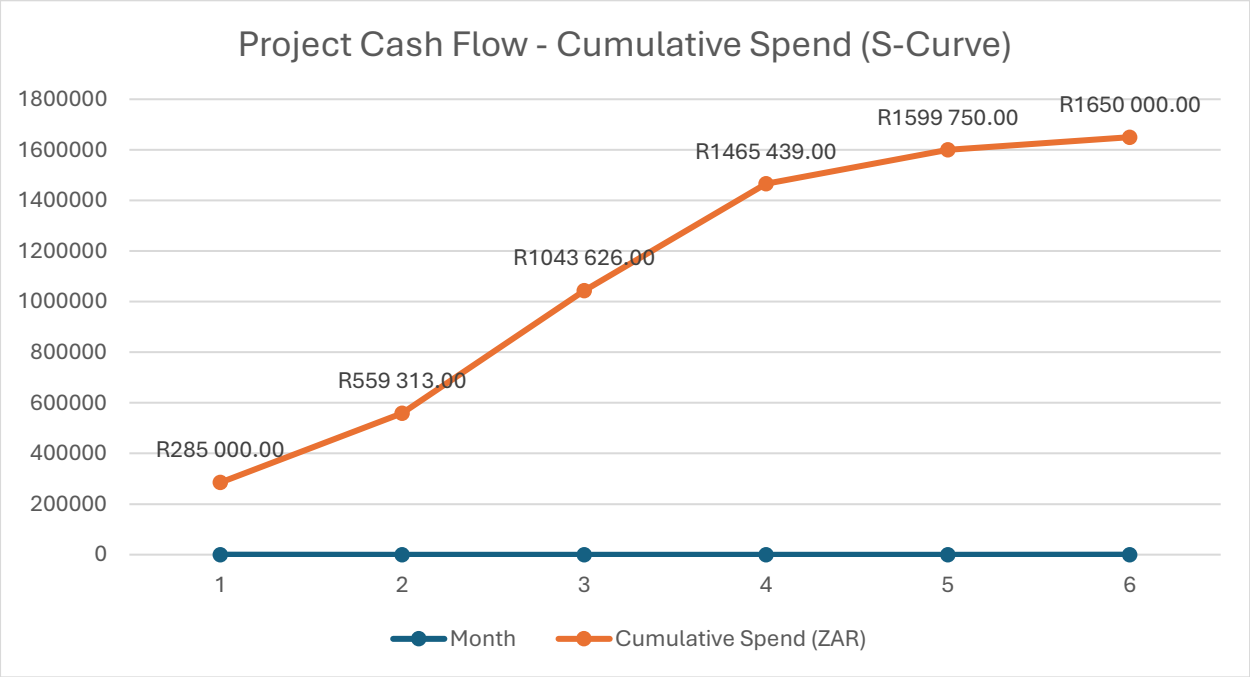
Appendix B: Cash Flow Projection

A monthly cash flow projection would show the cumulative spending over time.

Assumptions:

- Project Duration: 6 months (26 weeks).
- Labour Costs: Distributed monthly based on the project schedule (e.g., heavier costs during development months).
- Other Costs:
    - Contingency: Released as needed but planned evenly over the last 4 months.
    - Software/Licenses: Paid upfront in Month 1.
    - Training/Misc.: Incurred in the final month during the training and deployment phase.
- All values are in South African Rand (ZAR)

A	B	C	D	E	F
Month	**Phases in Progress**	**Labour (ZAR)**	**Other Costs (ZAR)**	**Monthly Total (ZAR)**	**Cumulative Spend (ZAR)**
1	Planning & Initiation	R195 000,00	R90 000,00	R285 000,00	R285 000,00
2	Design & Detailed Planning	R240 000,00	R34 313,00	R274 313,00	R559 313,00
3	Development Spring 1 & 2	R450 000,00	R34 313,00	R484 313,00	R1 043 626,00
4	Development Spring 3 & 4	R387 500,00	R34 313,00	R421 813,00	R1 465 439,00
5	Testing & Data Migration	R100 000,00	R34 311,00	R134 311,00	R1 599 750,00
6	Training, Deployment & Closure	R0,00	R50 250,00	R50 250,00	R1 650 000,00
	TOTAL	**R1 372 500,00**	**R277 500,00**	**R1 650 000,00**	



Appendix C: Responsibility Assignment Matrix (RAM)

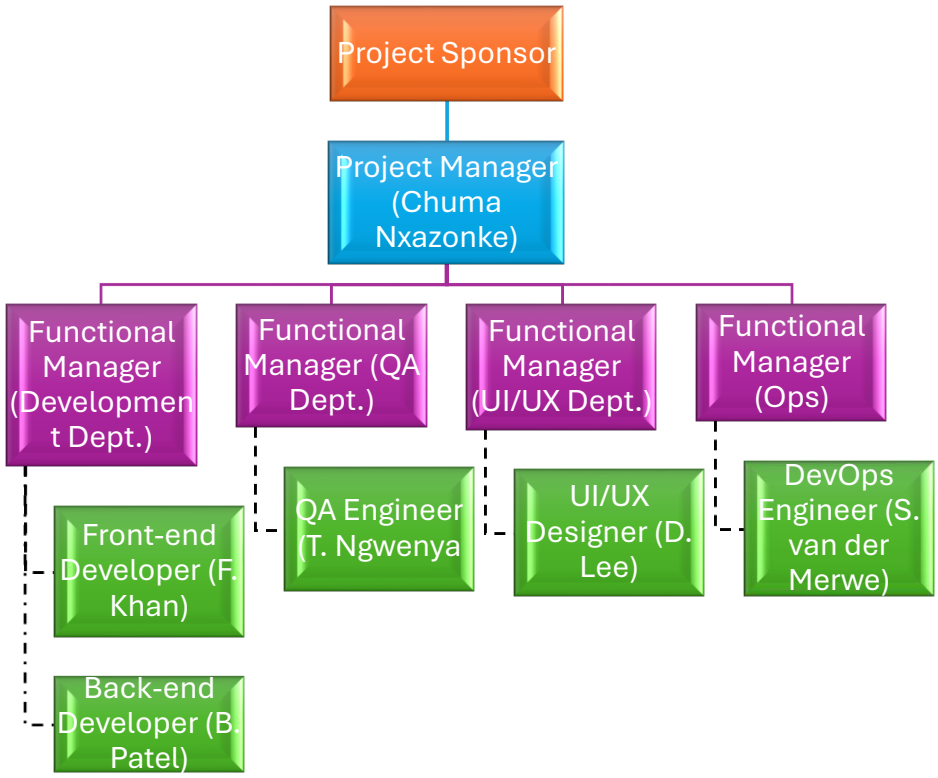
This matrix maps the project's key work packages to project roles, defining levels of responsibility using the RACI model.

Key: R - Responsible, **A** - Accountable, **C** - Consulted, **I** - Informed

Work Package / Activity	Project Manager	Business Analyst	UI/UX Designer	Back-end Developer	Front-end Developer	QA Engineer	DevOps Engineer
1. Planning & Design							
1.1 Project Kick-off	A	R	I	I	I	I	I
1.2 Requirements Gathering	A	R	C	C	I	I	I
1.3 Functional Specifications	A	R	C	C	C	I	I
1.4 UI/UX Design	A	C	R	I	C	I	I
1.5 Technical Architecture	A	I	I	R	C	I	C
2.Development							
2.1 User Auth Module	A	I	I	R	C	I	C
2.2 Sales Pipeline Module	A	C	I	R	R	I	I
2.3 Campaign Tracking Module	A	C	I	R	R	I	I
2.4 Reporting Dashboard	A	C	C	R	R	I	I
3. Testing							
3.1 Test Plan Creation	A	C	I	C	C	R	I
3.2 Execute Tests	I	I	I	I	I	R	I
4. Deployment							
4.1 CI/CD Pipeline Setup	A	I	I	C	C	C	R
4.2 Go-Live Deployment	A	I	I	I	I	I	R
5. Project Management							
5.1 Status Reporting	R/A	I	I	I	I	I	I
5.2 Risk Management	R/A	C	I	I	I	I	I

Figure 1: Project Organization Chart

The project's matrix organizational structure is illustrated in Figure 1 below, showing the dual reporting lines of team members to both the Project Manager and their Functional Managers.



Key: —— Solid Line: Direct Project Reporting

- - - - - Dotted Line: Functional Department Reporting

Figure 2: Resource Histogram - Project Team Allocation (2025)

This histogram illustrates the projected full-time team members allocated to the project per month, peaking during the development and testing phases (May-June) and tapering off during deployment and project closure (July-August).

Textual Data Table (for reference):

Month (2025)	Team Size (FTE)
March	3
April	6
May	7
June	7
July	5
August	2
Month (2025)	Team Size (FTE)